

Practices to make agile test teams effective: challenges and solutions

Tulasi Anand, V. S. Mani
Siemens Technology and Services Pvt. Ltd.
Bengaluru, India
tulasi.anand@siemens.com, vs.mani@siemens.com

Abstract— *this industrial practice paper presents the experience of a globally distributed test team of a software engineering organization spread across three countries (two in Europe and India) while moving to agile methodology. The team had to test mission-critical software that needed to conform to regulatory requirements. The paper describes the challenges faced by the team such as creating the test infrastructure in time, test automation, documentation and reporting mandated by regulatory agencies. The practices defined by the team to overcome these challenges are also described. These practices also helped to improve test effectiveness and velocity.*

Keywords—*test management, test infrastructure, test refactoring, test automation, agile, global software engineering, regulatory requirements.*

I. INTRODUCTION

The distributed teams worked on products, which enable users to manage confidential customer information along with acquisition, storage and management of images. The scope of test team is integration and system testing. The system under test also included several legacy applications that have inadequate testability. This made it unviable to achieve the objective of test sufficiency using test automation. This in turn increased manual testing effort.

The test teams accounted for about 20 per cent of overall project team. Each site has a team of test engineers who have about 4 to 12 years experience in the domain. The test engineers collaborate to achieve site specific test goals, derived from the overall test objectives. For the test activities that can be done at any location, the activity is assigned based on availability of test environment and competences. In some cases, the testing is done at a specific location, such as changes in software resulting from changes in firmware. In addition the changes in specific features are primarily tested at the location where the feature is developed.

The move to agile methodology was triggered by a need to improve both time-to-market and quality, while intensifying customer-centered development. There are already many experience reports of implementing Agile practices into global software development as given in [1],[2],[3]. The agile methodology followed a scrumming within the V-model paradigm [6].

II. CHALLENGES

Testing in agile is different from traditional software development due to the iterative nature of testing, shorter work

cycles, and increased dependence on collaboration [4], [5]. In our experience, these differences introduce new challenges which are listed below.

Infrastructure: It is difficult to quickly create a test execution environment in time, which ensures that the required customer-centric configurations are available to run the workflows under test. Customer-centric configurations are clones of the customer systems and consist of identical hardware in the customer site, operating system, third-party software, patches, and appropriate database. Another challenge is to install the versions of software under test on the customer-centric configurations.

Test Management: Executing the required test cases in the time provided in the sprints was a challenge and required identifying ways to more efficiently achieve the test objectives.

Regulatory agencies expect that all requirements in a product are traceable across all releases. This is especially challenging when the software product includes many legacy features and the development and testing of different components is done in multiple locations over time.

The rigor in following the planned site-specific test activities in the globally distributed agile team, made it difficult to execute exploratory test cases focused on parts of the software developed in other locations

Test automation: The software under test included of legacy code, with low testability. This made it difficult to automate even the bring-up test cases (see III-B-5 of this paper).

Test documentation: By test documentation we refer to the test concept, test specifications, test cases, test data, and test reports. The test documentation needs to be in synch with the features under development, which is especially challenging in globally distributed agile teams.

Knowledge Management: Knowledge transfer topic required close attention, as it was essential for the team to be up-skilled on legacy applications. These applications use older technology and methods and require longer time to learn and typically have a lower spread of knowledge.

III. PRACTICES

We have defined and implemented practices to address the challenges mentioned above.

A. Test infrastructure

1) Customer-centric configuration information

A practice of preparing an entirely virtualized test environment with backup ‘snapshots’ (a file-based representation of the state of a virtual machine at a given time) of different customer centric-configurations with anonymized data ensured the tests cases could be run, by testers at different locations at the same time. Despite this, some test cases had to be run at the customer site. For these test cases, service personnel were stationed near the customer.

In some instances, test cases had to be executed with actual equipment using different versions of firmware. In addition, legacy versions installed at different customer locations were not equally compatible with newer versions. These variations made getting the relevant information about the actual customer situation even more time consuming. We created a table containing details of the different customer-centric configurations and required equipment configurations. The table helped in creating and simulating end user workflows that effectively detected several defects that not uncommon but difficult to detect. Being able to detect them in the testing cycle significantly reduced the number of defects in the deployed applications.

2) Variety of test data

Usage of the application, over several releases of the product at a customer site, creates a large amount of data which is rich in variety and contains unforeseen and unexpected combinations with the possibility of incomplete data. In addition, the information needed by newer versions of the software may be missing. So a large number of pre-defined datasets were created to simulate both the missing information and data.

Use of anonymized datasets from various customer installations made it possible to visualize, manipulate and test the relevant workflow steps. This uncovered several defects, which would not have been detected otherwise. The volume and variety of data from actual customer installations has increased the confidence in testing significantly.

3) Software version upgrades and legacy versions

The software deployed at customer sites need to be upgraded to newer versions, without uninstalling the existing version or modifying the database. Multiple versions and patches deployed in the field make it difficult to determine the upgrade path for any version. The installation scripts specific to the different upgrade paths are created. Usage of customer centric data and configuration facilitated the upgrade tests as well. The different paths to reach the existing version were tested successfully in the test lab before the actual upgrade at the customer location. For the possible upgrade paths, usability of the application was tested before and after the upgrade.

4) Virtualization

Without virtualizing the entire test environment, it was not possible to effectively maintain the wide variety of versions, operating systems under test. Several customer-specific test environments were created in advance and kept ready for use when required. This made it easier to reproduce issues reported from customer sites. It also helped in defining and validating

fixes to address the issues. The same snapshot of the software was shared and was used for testing at different locations. The availability of correct configurations greatly improved the test results from different locations.

B. Test management and execution

1) Global Test planning

The global test plan is derived from the test concept. The test concept specifies the test approach, the major test assets required and their availability, and the strategy for confirming to regulatory requirements and acceptance with customers.

Defining the test concept requires a deep understanding of the domain, technology know-how, and awareness of the regulatory requirements.

2) Sprint/iteration level planning

The plan involving entire global team was prepared, results in test team coordination and stand up meetings covering different locations.

Some user stories were large and spanned multiple sprints. Scrum teams broke these down to smaller user stories and also defined the done criteria for these smaller user stories. This enabled assessing the degree of completion of planned tasks for the sprint and in coordinating activities.

3) Test only sprints and system testing cycles

To conform to the regulatory requirements, it was necessary to schedule a few ‘test-only’ sprints.

During the regular sprints, the test effort is about 15-18 percent of the total effort. However, this is not sufficient to ensure conformance to the regulatory requirements. Hence, after the regular sprints are completed, a few test-only sprints are scheduled. For the ‘test-only’ sprints, the test case selection is based on availability of fixes, impacted features, and new test cases created since the last testing cycle, new environment, and test data. In these sprints, complete regression testing at system level is done, as derived from the guidance from the regulatory agencies. In this phase all members of team are involved in testing. The approach is described in [6], recommended in reference [7].

4) Testing early

Before moving to agile, the number of integration tests was lower than the number of system tests. With the aim of executing more test cases early in the lifecycle, we tried to identify tests that could equivalently test the feature at an earlier phase. If it was possible for the objective of a system test case to be realized at the integration level, the system test case was moved to the integration test phase. The content of the test case was modified to reflect the change of methodology as well as the requirement tracing to the appropriate level. With this approach we were able to we were able to invert the distribution to test cases (see table 1).

These test cases were distributed and executed at different locations, by taking advantage of availability of appropriate test environment (equipment) and tester competences. Where ever possible, precise verification methods were relied upon to make the test result independent of the test engineer.

TABLE I. MOVEMENT OF TEST CASES TO EARLIER TEST PHASE

Phase	Proportion of test cases per phase	
	% of total test cases before moving to agile	% of total test cases after moving to agile
Integration testing	35	55
System testing	65	45

5) Bring-up tests

Once the nightly build is ready, it is taken up for detailed testing, only after it is assessed as test-worthy. This assessment is done by running ‘bring-up tests’. Bring-up tests are test cases that verify the most basic functionality of the application. Only when the bring-up tests are successfully run the build is tested. In addition, an automated test suite is run, which supplements the manual tests.

6) Regression suite

The application has various regression test suites, including functional and non-functional tests. Some regression tests are automated. However several regression tests are executed manually, such as simulating customer-centric configurations and testing for interoperability, using the approach of ‘Warm Body Testing’. This involves 12-15 test engineers, who simultaneously stress the system, to uncover workflow related issues. To ensure the efficiency, the ‘warm body testing’ was scheduled only when the application was sufficiently stable.

7) Focus on high impact errors

To make testing effective, every test engineer attempted to create the highest impact, by testing only as much as necessary. The code changes which had high impact on the entire functionality were prioritized, as they required running of entire test suite, which was time and effort intensive. For example, if the change resulted in a failure to save an object in a certain condition. The exact scenario where the failure occurs was identified. One single test procedure document containing all changes was created; rather than editing, reviewing, releasing all related documents containing small changes. This not only minimized the impact but also the test effort. The change was successfully tested and released. The proportion of defects detected as the sprints progressed is given in Fig. 2, showing a peak much before the last sprint and test-only sprints.

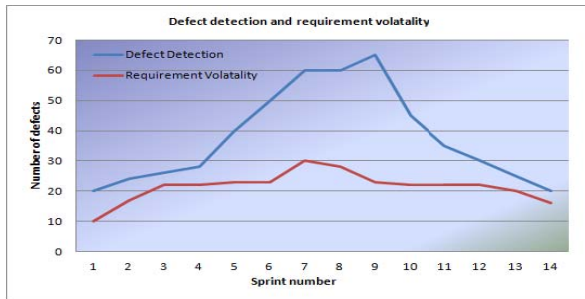


Fig. 1. Indicative defect detection during sprints

8) Exploratory testing

In the context of testing rapidly changing parts of requirements, exploratory testing techniques were used. The

development team provided relevant testable binaries created on a separate branch of code. The testers deployed on a specially created test environments, tested them in the context of the requirement change. The results of testing strengthened the requirement specification itself. Exploratory testing ended once the requirement was frozen. The related test scenarios were formalized as a regular test case in the test specification document.

9) Collaboration with stakeholders

The stake holders are [9]:

- *Product managers*: The collaboration has helped not only in evaluating the conformance to high level product requirements, but also in gaining valuable perspectives of product usability.
- *Senior management*: Regular meetings with them have given an insight in to product roadmap and future directions.
- *User manual authors*: Discussions with them have given an end-user view and this has helped in designing test cases, which are modeled along the same lines.
- *Application specialists*: Testing done by them, who actually train the end user at a customer site, has identified any wrong use case or understanding of the entire team. This has resulted in no workflow being rejected during acceptance testing.
- *Participation in trade shows* by test engineers has given an opportunity to interact with end-users directly and with other vendors

10) Involvement of customers

There are more options of products and competitors now than ever. Customers are getting technically savvy and are seeing value in increased use of specialized tools, both for development and testing. When done in moderation, it was possible for the team in Europe to interact and align with customers due to their proximity, on a regular basis seeking feedback through product demo and informal verification of fixes at customer sites. As mentioned in [11], controlled customer involvement is important an agile contribution, but attention must be paid to all stake holders.

C. Test automation

Automation was also used for testing non-functional requirements such as performance, reliability, stress. The automated performance tests using customer-centric configurations and end-user profiles. The performance was to exceed or match the performance of the previous version. In addition automated load tests ensured the system under test was able support the required number concurrent users.

The automation framework is designed at component interface level, rather than only at User interface level. The test automation test cases are run for every build before additional bring up tests that are executed manually.

Automated performance tests are run over several days has proved that the application can withstand the defined end-user

environments without defined ‘failure’ condition for a defined period of time. This suite was run on different customer centric test environments. This happens only in the test-only sprints, with the environment requirements being at a system level.

The allocation of test cases (including the performance test cases) to different location was done based on availability of test environment, expertise and proximity to equipment facility or to the customer.

D. Test document

In view of the large number of test cases, it is essential to concisely document test cases to be executed, which included: removing redundant test cases, merging similar test cases, removing repeated test steps or test cases, sequencing of test cases to remove redundancy of test execution, encapsulating the various test data input and state management as a table, where same test steps create different test scenarios by using different input conditions. This has reduced the effort to edit, review and maintain the test case.

1) Unambiguous defect reports

A wrong defect report can potentially lead to unproductive effort. Defect reports are discussed with development team, and senior testers before they are written, resulting in fewer defect reports being rejected. The test team participation in defect review meetings helped testers gain better perspective of impact of defects.

E. Knowledge management

1) Exposure to testers

Test engineers need to have application domain training/exposure to end user workflows, should be aware of competitor’s solutions, and establish/improve communication with Product Management. Testers were made aware of the safety related aspects [10], and the extraordinarily high cost of failure. A single defect could be fatal, like in application domain like ours.

Training provided in application workflows, customer site visits, weekly meeting with customer care personnel have helped our test case design.

2) Applying learning

Due to complexities of technology, design, legacy, errors can get introduced in multiple ways. Therefore, testers should try to force all possible failures, by prioritizing the test cases to execute in the limited time frames. Testers added business value, by verifying the functionality, with a focus on reducing waste and rework. Testers attended courses that focus on research based topics like orthogonal array testing. Applying leanings from conferences, trade shows in to day-to-day enhanced value testers provided.

3) Increasing quality focus

According to Gartner Research [11], “The lack of testing and QA standards, as well as a lack of consistency, often lead to business disruption, which can be costly.” Gartner also reports that “testing consumes 25% to 50% of the average application life cycle and often is viewed as adding no business value”. Adherence to quality, regulatory and organization quality standards is essential to reduce unproductive effort spent on testing. Recognizing the need, testers are provided the

same access to team activities, portals and documentation as other team members.

4) Knowledge acquisition

It typically takes a long time to learn the complex domain of our application. The complexity is in the terminology, conceptual understanding and a wide range of interfacing systems. A training plan involving all the basics was imparted through instructor-led trainings, supplemented with online training material.

5) Training related regulatory requirements

The regulatory environment required not only a test result in terms of ‘passed/failed’ status that is traced to a requirement, but a verifiable demonstration of functionality (test outcome), explicit in the recorded evidence of the test execution. A training provided on collection of appropriate test evidence has satisfied the regulatory requirement.

A rigorous enforcement of requirement tracing and collecting evidence of testing through screen shots of results and logs during test execution, clear document versioning, keeping a record of document reviews helped in complying with regulatory requirements.

6) Competency development

Agile requires that test engineers ‘become generalizing specialists’ [4]. To achieve this, competency development plans were defined that listed the training needs and people who had the know-how to provide the training. Several audio and video recordings of the past trainings were made available to the test engineers, who could get detailed information on the topics without requiring any instructor support. This was followed by an instructor led training, and then by ‘shadowing’ and ‘reverse-shadowing’. Once the test engineer acquired the additional knowledge, he/she was assigned independent charge of a specific area. To ensure that a skilled engineer was always available, typically two test engineers were trained on each topic.

7) Training for testers on test case design

We defined a training program [12] that contained topics on test case design, test management and technical test analyst. Identified test engineers were trained on this program, by senior test team members. Briefly test case design techniques are listed below:

- Use of flow charts and tables was encouraged during test case design.
- A set of “do’s-don’ts” have helped in a uniform understanding of the test case design methodology.
- Test objective was stated precisely matching the requirements being tested and not as generic statements, which don’t imply anything specific.
- Risk mitigation test cases were treated as a separate set of test cases to give better focus.
- The dataflow test cases identified and recorded all application attributes and their combinations that would be passed through, during test execution

- The control flow test cases covered the test cases that have a result dependent on the path traversed than the dataset conditions. Both linear and non-linear control paths are executed to give complete test execution coverage.
- The other set of test cases covered the dependency on the sequencing of individual test steps.
- The test on the User Interface elements dependent on all the above test scenarios were recorded and tested in conjunction.
- Test cases for boundary conditions, equivalence partitioning, error guessing were included as a standard group in the test suite.

IV. SUMMARY AND CONCLUSION

Summary of the challenges and measure is given below. The testing practices for enabling the global team moving to agile have been successful. The team has earned the confidence of the business unit in the effectiveness of testing in globally distributed organization. The velocity has consistently been improving over the years and at the same time has improved defect detection. The rigor in these practices helped manage the expectations.

REFERENCES

- [1] Mario Ivček, Tihana Galinac, "Adapting Agile practices in globally distributed large scale software development" MIPRO 2009 - 32nd International Convention Proceedings: Telecommunications and Information Conference Paper
- [2] Jasper Goos, Alfo Melisse. "An Ericsson Example of Enterprise Class Agility" Agile 2008 Conference.
- [3] Paasivaara, M., Lassenius, C., Heikkila, V.T., Dikert, K., Engblom, C., "Integrating Global Sites into the Lean and Agile Transformation at Ericsson" 2013 IEEE 8th International Conference on Global Software Engineering
- [4] Agile Testing and Quality Strategies: Discipline Over Rhetoric, Ambysoft, <http://www.ambysoft.com/essays/agileTesting.html>
- [5] Scott Ambler, Mark Lines, "Disciplined Agile Delivery, A Practitioner's Guide to Agile Software Delivery in the Enterprise " IBM Press, 2012
- [6] Managing Requirements Volatility while 'Scrumming' within the V Model by Anita PC, Deepti Savio, VS Mani
- [7] Adopting Agile in an FDA, Regulated Environment' [Abbott 2009], Abbot Labs
- [8] Software Design at Any Price, Interview with Thomas Price for the 2013 Software Design for Medical Devices Conference
- [9] Stakeholders in Testing – Tulasi Anand, Siemens SWI – Exchange of Experience- Testing, 2008
- [10] Agile! The Good, the Hype and the Ugly, Published by Springer, ISBN 978-3-319-05155-0 (eBook), by Bertrand Meyer
- [11] Partha Iyengar, Frances Karamouzis. "Offshore Application Testing Drives Greater Business Value." Gartner Research, August 17, 2007, ID Number G00150394
- [12] Samartham, G., Suryanarayana, G. Gupta, A.K., Nambiar, R., "FOCUS: An adaptation of a SWEBOK-based curriculum for industry requirements." 34th International Conference on Software Engineering (ICSE), 2012